Herbal Health Seminar Series

Product Specification & Technical Requirements Document

Document version: 1.0

Date: 2026-09-10

Status: Developer-ready baseline specification

Reference implementation: Next.js full-stack application, Prisma ORM, PostgreSQL

Primary objective: Rebuild and improve the existing Herbal Health Seminar Series website as a maintainable event discovery, registration, ticketing, payment, and administration platform.

---

1. Executive Summary

The existing Herbal Health Seminar Series website is a marketing and event-registration website for traveling educational herbal-health seminars led by Dr. JC Goodman, N.D. The public experience centers on discovering seminars, reading event details, selecting tickets, and proceeding to checkout.

The rebuild should preserve the current business model while replacing the Wix-dependent implementation with a custom Next.js application. The application will have:

1. A public SEO-friendly website.
2. A dynamic event/seminar catalog.
3. Event detail pages generated from database content.
4. Ticketing and registration.
5. Payment processing through a pluggable payment provider.
6. Attendee and order management.
7. An administrative dashboard/CMS.
8. Email notifications and event reminders.
9. Legal/disclaimer acknowledgement.
10. Reporting, audit logging, and operational controls.

The system should be designed as a reusable seminar platform rather than as a collection of hard-coded pages.

Current-site observations

The live site currently presents the seminar series, mission, upcoming events, educational themes, FAQs, and registration CTAs. Its current event catalog includes Tour 1 in Orlando and future Volumes 2 and 3 in Charlotte and Atlanta. Tour 1 has a two-day schedule, a ticket price of \$1,500, a ticket-sale deadline, and a checkout flow. The event page

also contains an educational-purpose/medical-disclaimer statement.

Reference pages reviewed: - https://www.herbalhealth.online/ -

https://www.herbalhealth.online/event-list -

https://www.herbalhealth.online/event-details/herbal-health-seminar-series-tour-1-foundations-everyday-herbal-remedies

---

2. Product Vision

Build a premium, reliable, conversion-focused seminar platform that

allows Herbal Health Seminar Series to publish educational events, sell

registrations, collect payments, communicate with attendees, and manage

the complete event lifecycle from one administrative system.

Product principle

> Create the event once in the admin dashboard; the public website,

> ticket inventory, registration, checkout, confirmation, and reporting

> should derive from that single source of truth.

---

3. Goals

3.1 Primary goals

• Replace Wix event management with a custom application.

• Preserve the current public website’s core content and user journey.

• Improve performance, SEO, accessibility, and maintainability.

• Make events fully database-driven.

• Allow non-developers to create and manage events.

• Support multiple ticket types per event.

• Support payment status tracking.

• Prevent overselling.

• Provide reliable registration confirmation.

• Provide attendee management.

• Provide event reminders.

• Provide legal/disclaimer acknowledgement.

• Provide auditability for important administrative actions.

3.2 Secondary goals

• Support future courses and digital products.

• Support coupons/discounts.

• Support waitlists.

• Support certificates.

• Support multiple instructors.

• Support multiple venues/cities.

• Support analytics and conversion tracking.

3.3 Non-goals for MVP

• Full learning-management system.

• Medical consultation system.

• Patient records.

• Prescription management.

• Telemedicine.

• Herbal product marketplace.

• Social network/community platform.

• Native mobile application.

---

4. Target Users

4.1 Public visitor

Can: - View the homepage. - Browse seminars. - Search/filter events. - Read event information. - Read FAQs. - Read instructor information. - Contact the organization. - Register for an event.

4.2 Attendee

Can: - Complete registration. - Pay for tickets. - Receive confirmation. - Access registration/ticket information. - Receive event reminders. - Request support. - Receive cancellation/refund notifications.

MVP does not require a persistent attendee account. Registration may be guest-based using email plus a secure registration reference.

4.3 Administrator

Can: - Manage events. - Manage tickets. - Manage attendees. - Manage orders. - Manage payments. - Manage content. - Manage FAQs. - Manage instructors. - Manage locations. - Send notifications. - View reports. - Manage settings. - View audit logs.

4.4 Content editor

Can: - Edit pages. - Edit FAQs. - Edit event descriptions. - Upload images. - Manage educational content.

Content editors should not automatically receive financial/refund permissions.

---

5. Roles & Permissions

Capability	Admin	Content Editor
View dashboard	Yes	Yes
Create/edit events	Yes	Yes
Publish events	Yes	Optional
Manage ticket prices	Yes	No
View payments	Yes	No
Process refunds	Yes	No
View attendee PII	Yes	Limited
Export attendees	Yes	No
Manage FAQs	Yes	Yes
Manage pages	Yes	Yes
Manage instructors	Yes	Yes
Manage locations	Yes	Yes
Manage users/roles	Yes	No
View audit logs	Yes	No
Change system settings	Yes	No

---

6. Functional Architecture

```
PUBLIC WEB
|
Next.js App Router
|
+---------------+----------------+
| | |
Pages Events Registration
| | |
+---------------+----------------+
|
Next.js Server
Route Handlers / Server
Actions / Services
|
+---------------+----------------+
| | |
Prisma Auth/RBAC Payments
| | |
+---------------+----------------+
|
PostgreSQL
|
+----------+----------+
| |
Email Storage
Provider/API Image/Documents
```

---

7. Recommended Technology Stack

7.1 Application

• Next.js

• TypeScript

• React

• App Router

• Server Components by default

• Server Actions where appropriate

• Route Handlers for public/webhook APIs

7.2 Database

• PostgreSQL

• Prisma ORM

• Prisma Migrate

• Prisma generated client

7.3 Authentication

Recommended: - Auth.js/NextAuth or an equivalent secure authentication solution. - Admin-only authentication. - Email/password or organization-approved identity provider. - Optional MFA in a future increment.

7.4 Validation

• Zod for request and form validation.

• Shared schemas between UI and API where practical.

7.5 Styling/UI

Recommended: - Tailwind CSS. - Accessible component system such as shadcn/ui or equivalent. - Responsive-first design.

7.6 Email

Provider abstraction with: - Transactional email provider. - HTML and text templates. - Delivery status logging.

7.7 File storage

Object storage abstraction for: - Event images. - Instructor images. - Downloadable documents. - Future attendee materials.

7.8 Payments

Payment-provider adapter:

```
interface PaymentProvider {
createCheckoutSession(input: CheckoutInput): Promise<CheckoutSession>;
verifyWebhook(payload: string, signature: string): Promise<WebhookEvent>;
refund(input: RefundInput): Promise<RefundResult>;
}
```

The provider must be selected before implementation. The application must not tightly couple core business logic to one payment vendor.

---

8. Public Website Information Architecture

Required routes

```
/
/events
/events/[slug]
/about
/instructor/[slug]
/faq
/contact
/terms
/privacy
/refund-policy
/disclaimer
/checkout/[session]
/registration/success
/registration/[reference]
```

Optional future routes

```
/courses
/resources
/blog
/certificates
/account
```

---

9. Homepage Requirements

The homepage should contain:

1. Header/navigation.

2. Hero section.

3. Primary CTA: View Upcoming Seminars / Register.

4. Mission section.

5. Upcoming seminar section.

6. Educational focus sections.

7. Instructor section.

8. FAQ section.

9. Final registration CTA.

10. Footer.

11. Legal links.

Homepage data should be CMS-driven

The administrator should be able to change: - Hero heading. - Hero text. - CTA. - Mission copy. - Educational focus items. - Featured events. - FAQ items. - Footer content.

---

10. Event Catalog

10.1 Event listing

Each event card should display:

• Cover image.

• Event title.

• Volume/tour label.

• Date.

• City/state.

• Short description.

• Event status.

• CTA.

Status-driven CTA

Event state	CTA
Draft	Not public
Coming Soon	View Details
Published / Not on sale	Details
On Sale	Buy Tickets
Sold Out	Join Waitlist
Closed	Event Closed
Cancelled	Cancelled
Completed	View Recap

---

11. Event Model

Each event must support:

Identity

• ID

• Slug

• Internal code

• Title

• Series name

• Volume/tour number

• Short description

• Long description

Schedule

• Start date/time

• End date/time

• Timezone

• Optional multiple sessions

Location

• Venue name

• Address

• City

• State

• Postal code

• Country

• Latitude/longitude

• Optional map URL

Content

• What attendees will learn

• Materials included

• Prerequisites

• Instructor(s)

• Disclaimer

• FAQ overrides

Commerce

• Ticket types

• Price

• Currency

• Capacity

• Sales start

• Sales end

• Refund policy

Media

• Hero image

• Gallery

• Instructor image references

• Downloadable documents

Publication

• Draft

• Published

• Coming Soon

• On Sale

• Sold Out

• Closed

• Cancelled

• Completed

---

12. Event Creation Workflow

Admin selects:

Events → Create Event

Required fields:

• Title.

• Slug.

• Short description.

• Full description.

• Start/end date.

• Timezone.

• Location.

• Cover image.

• Event status.

Then admin adds:

• Learning objectives.

• Materials.

• Instructors.

• Ticket types.

• Capacity.

• Sales window.

• Disclaimer.

Workflow:

```
Draft
↓
Preview
↓
Publish
↓
On Sale
↓
Sold Out / Closed
↓
Completed
```

Publishing validation must prevent incomplete public events.

---

13. Ticketing

13.1 Ticket type

Fields:

• ID

• Event ID

• Name

• Description

• Price

• Currency

• Quantity available

• Quantity sold

• Maximum quantity per order

• Sale start

• Sale end

• Status

13.2 Ticket inventory

The system must prevent overselling under concurrent requests.

Preferred approach:

• Use database transaction.

• Lock or atomically update inventory.

• Create an order/payment session only after inventory reservation.

• Reservation expires if payment is not completed.

Inventory states

```
AVAILABLE
LOW
SOLD_OUT
CLOSED
```

---

14. Cart / Checkout

MVP can use a simplified checkout without a persistent cart.

Flow:

```
Event
↓
Select ticket type
↓
Select quantity
↓
Enter attendee details
↓
Accept legal terms/disclaimer
↓
Create pending order
↓
Reserve inventory
↓
Create payment session
↓
Payment provider
↓
Webhook
↓
Mark order paid
↓
Create/confirm registration
↓
Send confirmation
```

Required attendee fields:

• First name

• Last name

• Email

• Phone

• Country

• Optional organization

• Optional accessibility/request notes

Do not collect unnecessary health information.

---

15. Orders

Order status:

```
PENDING
PAYMENT_PROCESSING
PAID
```

```
FAILED
CANCELLED
REFUNDED
PARTIALLY_REFUNDED
EXPIRED
```

Order should contain:

• Order number.

• User/attendee reference if applicable.

• Currency.

• Subtotal.

• Discount.

• Tax/fee fields if applicable.

• Total.

• Payment provider.

• Payment reference.

• Created timestamp.

• Updated timestamp.

Money must be stored as integer minor units, not floating point.

Example:

```
$1,500.00 -> 150000 cents
```

---

16. Registration

A successful paid order produces one or more registrations.

Registration fields:

• Registration ID.

• Public reference.

• Order ID.

• Event ID.

• Ticket type.

• Attendee.

• Status.

• Check-in status.

• Registration date.

Registration statuses:

```
CONFIRMED
CANCELLED
REFUNDED
TRANSFERRED
ATTENDED
NO_SHOW
```

---

17. Attendee Management

Admin attendee table:

Field	Description
Name	Attendee name
Email	Contact email
Phone	Contact phone
Event	Registered event
Ticket	Ticket type
Payment	Paid/unpaid/refunded
Registration	Current status
Check-in	Not checked in / checked in
Registered	Timestamp

Admin actions:

• View attendee.

• Resend confirmation.

• Cancel registration.

• Request/refund payment.

• Transfer registration.

• Mark attended.

• Export attendee.

• Add internal note.

---

18. Registration Reference

Each confirmed registration receives a human-readable reference, e.g.:

```
HHS-2026-8F4K2Q
```

Reference must be: - Unique. - Non-sequential enough to avoid easy enumeration. - Safe to expose publicly.

---

19. Ticket / QR Code

Future-ready architecture should support QR codes.

QR payload should contain an opaque registration identifier, not sensitive personal information.

Example:

```
registration:HHS-2026-8F4K2Q
```

Admin check-in flow:

```
Scan QR
↓
Validate registration
↓
Verify event
↓
Verify paid/confirmed status
↓
Mark checked in
```

---

20. Payment Architecture

Payment processing must be webhook-driven.

Do not trust only the client redirect to confirm payment.

```
Client
↓
Create order
↓
Payment session
↓
Payment provider
↓
Provider webhook
↓
Verify signature
↓
Find order
↓
Idempotency check
↓
Update payment
↓
Update order
↓
Confirm registration
↓
Send confirmation
```

Webhook requirements

• Signature verification.

• Idempotency.

• Transactional updates.

• Event logging.

• Retry-safe processing.

A repeated webhook must not create duplicate registrations.

---

21. Refunds

Admin can request a refund for eligible orders.

Refund flow:

```
Admin
↓
Select order
↓
Select refund amount/reason
↓
Confirm
↓
Payment provider refund
↓
Record refund
↓
Update order
↓
Update registration
↓
Notify attendee
```

Refund rules should be configurable per event or globally.

---

22. Legal & Disclaimer Requirements

The site is educational and must clearly distinguish educational content

from medical advice.

The current website explicitly states that the seminar is for

educational purposes and does not provide medical advice, diagnosis, treatment, or prescribe remedies.

The rebuild must include:

- Global disclaimer page.
- Event disclaimer.
- Terms and conditions.
- Privacy policy.
- Refund/cancellation policy.
- Registration acknowledgement.
- Timestamp of acceptance.
- Version of accepted terms/disclaimer.

At checkout:

```
[ ] I have read and agree to the Terms & Conditions and acknowledge the educational disclaimer.
```

The registration cannot be completed without required acknowledgement.

Legal copy should be reviewed by the organization's legal adviser before production.

---

23. Content Management

The admin should be able to manage:

- Homepage.
- About page.
- Instructor profiles.
- Events.
- FAQs.
- Footer.
- Legal pages.
- Educational focus blocks.

Avoid building a general-purpose CMS in MVP. Use structured content models first.

---

24. FAQ System

FAQ entity:

- ID
- Question
- Answer
- Category
- Sort order
- Published
- Created at

• Updated at

Admin can: - Create. - Edit. - Delete/archive. - Reorder. -

Publish/unpublish.

---

25. Instructor Management

Instructor fields:

• Name.

• Credentials.

• Short bio.

• Full bio.

• Photo.

• Website/social links if approved.

• Published status.

Events may have one or multiple instructors.

---

26. Location Management

Reusable location entity:

• Venue name.

• Address.

• City.

• State.

• Postal code.

• Country.

• Timezone.

• Latitude.

• Longitude.

• Map URL.

• Parking instructions.

• Venue notes.

Events reference locations rather than duplicating location data.

---

27. Email Notifications

Required emails:

1. Registration confirmation.

2. Payment receipt.

3. Payment failure.

4. Event reminder.

5. Event location/update notification.

6. Event cancellation.

7. Refund confirmation.

8. Registration transfer.

9. Admin notification for new registration.

Email templates should support variables:

```
{{attendee.firstName}}
{{event.title}}
{{event.startDate}}
{{event.location}}
{{registration.reference}}
{{order.total}}
```

Email sending should be asynchronous where practical.

---

28. Reminder System

MVP:

• 7 days before.

• 1 day before.

Future: - 30 days before. - 3 hours before. - Custom reminders.

Reminder jobs must be idempotent.

---

29. Admin Dashboard

Dashboard widgets

• Upcoming events.

• Total registrations.

• Revenue.

• Pending payments.

• Sold-out events.

• Recent registrations.

• Upcoming event capacity.

Events

• List.

• Create.

• Edit.

• Duplicate.

• Preview.

• Publish/unpublish.

• Close sales.

• Cancel.

• Complete.

Orders

• Search.

• Filter.

• View.

• Refund.

• Export.

Attendees

• Search.

• Filter.

• Check-in.

• Export.

Content

• Pages.

• FAQs.

• Instructors.

• Locations.

Reports

• Event registrations.

• Revenue.

• Ticket sales.

• Attendance.

• Refunds.

---

30. Admin Event List

Columns:

• Event.

• Location.

• Date.

• Status.

• Tickets sold.

• Capacity.

• Revenue.

• Actions.

Filters:

• Status.

• City.

• Date range.

• Sales status.

---

31. Admin Event Editor

Use sections/tabs:

```
General
Schedule
Location
Content
Learning Objectives
Instructors
Tickets
Media
SEO
Legal
Publication
```

Autosave is optional. Explicit Save/Publish should remain available.

---

32. Search & Filtering

Public event catalog should support:

• Keyword.

• City.

• State.

• Date range.

• Event status.

MVP can use PostgreSQL queries. Avoid introducing

Elasticsearch/OpenSearch until scale requires it.

---

33. SEO

Every public event should have:

• SEO title.

• Meta description.

• Canonical URL.

• Open Graph image.

• Structured data.

Recommended schema:

• `Event`

• `Organization`

• `Person`

• `FAQPage` where appropriate.

Generate:

```
/sitemap.xml
/robots.txt
```

Only published public content should enter the sitemap.

---

34. Accessibility

Target WCAG 2.2 AA.

Requirements:

• Keyboard navigation.

• Visible focus states.

• Semantic HTML.

• Form labels.

• Accessible validation errors.

• Sufficient contrast.

• Alt text.

• Screen-reader-friendly dialogs.

• Accessible date/time inputs.

• No color-only status indicators.

---

35. Performance

Targets:

• Fast initial page load.

• Server-render public pages where possible.

• Optimize images.

• Lazy-load below-the-fold images.

• Minimize client JavaScript.

• Cache public event content where appropriate.

Next.js Server Components should be the default.

---

36. Security

Required:

• HTTPS in production.

• Secure cookies.

• CSRF protection where applicable.

• Rate limiting on sensitive endpoints.

• Input validation.

• Output encoding.

• SQL injection protection through Prisma parameterization.

• Authorization checks on every admin operation.

• Webhook signature verification.

• Audit logging.

• No payment-card storage.

• No sensitive payment credentials in database.

• Secrets only in environment variables/secret manager.

---

37. Privacy

Collect only data necessary to:

• Process registration.

• Process payment.

• Communicate about events.

• Provide support.

• Meet legal/accounting requirements.

Admin exports containing PII should be permission-controlled.

---

38. Audit Log

Record important administrative actions:

• Event created.

• Event edited.

• Event published.

• Event cancelled.

• Ticket price changed.

• Ticket inventory changed.

• Order refunded.

• Registration cancelled.

• Attendee data changed.

• User role changed.

Audit fields:

• Actor.

• Action.

• Entity type.

• Entity ID.

• Timestamp.

• IP where appropriate.

• Before/after metadata where safe.

---

39. Database Design

Core models

```
AdminUser
Role
Event
EventSession
Location
Instructor
EventInstructor
LearningObjective
TicketType
```

```
Order
OrderItem
Payment
Refund
Attendee
Registration
RegistrationTermAcceptance
FAQ
ContentPage
MediaAsset
Coupon
Notification
EmailLog
AuditLog
```

Relationship overview

```
Event
■■■ Location
■■■ EventSession[]
■■■ EventInstructor[] ■■ Instructor
■■■ LearningObjective[]
■■■ TicketType[]
■■■ Registration[]
■■■ MediaAsset[]

Order
■■■ OrderItem[]
■■■ Payment[]
■■■ Refund[]
■■■ Attendee/Registration

Attendee
■■■ Registration[]
```

---

40. Prisma Schema Baseline

Illustrative baseline; developers should refine indexes and provider-specific types during implementation.

```
enum EventStatus {
DRAFT
COMING_SOON
PUBLISHED
ON_SALE
SOLD_OUT
CLOSED
CANCELLED
COMPLETED
}

enum OrderStatus {
PENDING
PAYMENT_PROCESSING
PAID
FAILED
CANCELLED
REFUNDED
PARTIALLY_REFUNDED
EXPIRED
}

enum RegistrationStatus {
CONFIRMED
CANCELLED
REFUNDED
TRANSFERRED
ATTENDED
NO_SHOW
}

enum PaymentStatus {
PENDING
PROCESSING
SUCCEEDED
FAILED
REFUNDED
PARTIALLY_REFUNDED
}
```

```
model Event {
id String @id @default(cuid())
slug String @unique
title String
seriesName String?
volumeLabel String?
shortDescription String
description String
status EventStatus @default(DRAFT)
startAt DateTime
endAt DateTime
timezone String
capacity Int?
salesStartAt DateTime?
salesEndAt DateTime?
locationId String
disclaimer String?
seoTitle String?
seoDescription String?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

location Location @relation(fields: [locationId], references: [id])
sessions EventSession[]
tickets TicketType[]
registrations Registration[]
instructors EventInstructor[]
objectives LearningObjective[]
}

model Location {
id String @id @default(cuid())
venueName String?
address1 String
address2 String?
city String
state String?
postalCode String?
country String
timezone String
mapUrl String?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

events Event[]
}

model TicketType {
id String @id @default(cuid())
eventId String
name String
description String?
priceMinor Int
currency String
capacity Int?
maxPerOrder Int @default(10)
salesStartAt DateTime?
salesEndAt DateTime?
active Boolean @default(true)
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

event Event @relation(fields: [eventId], references: [id])
orderItems OrderItem[]
}

model Attendee {
id String @id @default(cuid())
firstName String
lastName String
email String
phone String?
country String?
organization String?
notes String?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

orders Order[]
registrations Registration[]
}

model Order {
id String @id @default(cuid())
```

```
  orderNumber String @unique
  attendeeId String?
  status OrderStatus @default(PENDING)
  currency String
  subtotalMinor Int
  discountMinor Int @default(0)
  totalMinor Int
  paymentProvider String?
  providerRef String?
  expiresAt DateTime?
  paidAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  attendee Attendee? @relation(fields: [attendeeId], references: [id])
  items OrderItem[]
  payments Payment[]
  refunds Refund[]
  registrations Registration[]
  }

  model OrderItem {
  id String @id @default(cuid())
  orderId String
  ticketTypeId String
  quantity Int
  unitPriceMinor Int
  totalMinor Int

  order Order @relation(fields: [orderId], references: [id])
  ticketType TicketType @relation(fields: [ticketTypeId], references: [id])
  }

  model Payment {
  id String @id @default(cuid())
  orderId String
  provider String
  providerRef String?
  status PaymentStatus @default(PENDING)
  amountMinor Int
  currency String
  rawEventId String?
  paidAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  order Order @relation(fields: [orderId], references: [id])

  @@index([provider, providerRef])
  }

  model Registration {
  id String @id @default(cuid())
  reference String @unique
  orderId String
  eventId String
  attendeeId String
  ticketTypeId String
  status RegistrationStatus @default(CONFIRMED)
  checkedInAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  order Order @relation(fields: [orderId], references: [id])
  event Event @relation(fields: [eventId], references: [id])
  attendee Attendee @relation(fields: [attendeeId], references: [id])
  ticketType TicketType @relation(fields: [ticketTypeId], references: [id])
  termsAcceptances RegistrationTermAcceptance[]
  }

  model RegistrationTermAcceptance {
  id String @id @default(cuid())
  registrationId String
  documentType String
  version String
  acceptedAt DateTime
  ipAddress String?

  registration Registration @relation(fields: [registrationId], references: [id])
  }
```

---

41. API Architecture

Because Next.js is being used for both UI and API, use:

• Server Actions for internal form mutations where appropriate.

• Route Handlers for webhooks and externally consumable endpoints.

• Service layer for business logic.

• Prisma repository/data-access layer.

Recommended structure:

```
src/
app/
(public)/
admin/
api/
payments/
webhooks/
registrations/
components/
lib/
auth/
db/
payments/
email/
validation/
server/
services/
repositories/
prisma/
schema.prisma
```

---

42. API Endpoints

Public

```
GET /api/events
GET /api/events/:slug
GET /api/events/:slug/tickets
POST /api/checkout
GET /api/registrations/:reference
POST /api/waitlist
```

Admin

```
GET /api/admin/events
POST /api/admin/events
GET /api/admin/events/:id
PATCH /api/admin/events/:id
DELETE /api/admin/events/:id

POST /api/admin/events/:id/publish
POST /api/admin/events/:id/close
POST /api/admin/events/:id/cancel

GET /api/admin/orders
GET /api/admin/orders/:id
POST /api/admin/orders/:id/refund

GET /api/admin/attendees
GET /api/admin/registrations
POST /api/admin/registrations/:id/check-in

GET /api/admin/reports/revenue
GET /api/admin/reports/attendance
```

Exact API exposure should be minimized; internal admin operations can

use Server Actions.

---

43. Service Layer

Business rules should not live directly in React components.

Recommended services:

```
EventService
TicketService
InventoryService
CheckoutService
OrderService
PaymentService
RegistrationService
RefundService
NotificationService
EmailService
ContentService
ReportService
AuditService
```

Example:

```
CheckoutService.createCheckout()
PaymentService.handleWebhook()
RegistrationService.confirmFromPaidOrder()
InventoryService.reserve()
InventoryService.release()
```

---

44. Transaction Boundaries

Critical operations must be transactional.

Confirm payment

```
BEGIN
lock/read payment/order
verify not already processed
mark payment succeeded
mark order paid
create registrations
finalize inventory
COMMIT
```

If email sending fails, the payment transaction should still succeed.

Email should be retried separately.

---

45. Inventory Reservation

Recommended MVP implementation:

1. User submits checkout.

2. Begin transaction.

3. Validate ticket availability.

4. Reserve inventory.

5. Create pending order.

6. Create payment session.

7. Commit.

8. If payment is not completed before expiry, release reservation.

A scheduled cleanup job should release expired reservations.

---

46. Idempotency

Required for:

• Payment webhooks.

• Checkout creation.

• Refunds.

• Confirmation emails.

• Reminder emails.

Store provider event IDs and/or idempotency keys.

---

47. Background Jobs

The MVP requires a mechanism for scheduled jobs.

Tasks:

• Expire pending orders.

• Release ticket reservations.

• Send event reminders.

• Retry failed emails.

• Generate reports if needed.

Implementation options:

• Vercel Cron + database-backed job table.

• External queue in future.

The first implementation should remain simple and observable.

---

48. Error Handling

Use structured application errors.

Categories:

```
VALIDATION_ERROR
AUTHENTICATION_ERROR
AUTHORIZATION_ERROR
NOT_FOUND
CONFLICT
INVENTORY_UNAVAILABLE
PAYMENT_ERROR
EXTERNAL_SERVICE_ERROR
RATE_LIMITED
INTERNAL_ERROR
```

User-facing messages must not expose stack traces or sensitive provider

information.

---

49. Observability

Production should include:

• Structured logs.

• Error tracking.

• Payment webhook logs.

• Email delivery logs.

• Audit logs.

• Request correlation IDs where practical.

Monitor:

• Checkout failures.

• Payment failures.

• Webhook failures.

• Registration failures.

• Email failures.

• Database errors.

---

50. Environment Variables

Example:

```
DATABASE_URL=
DIRECT_URL=

AUTH_SECRET=

NEXT_PUBLIC_SITE_URL=

PAYMENT_PROVIDER=
PAYMENT_SECRET_KEY=
PAYMENT_WEBHOOK_SECRET=

EMAIL_PROVIDER=
EMAIL_API_KEY=
EMAIL_FROM=

STORAGE_PROVIDER=
STORAGE_BUCKET=
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=

CRON_SECRET=
```

Never commit secrets to source control.

---

51. Testing Requirements

Unit tests

Test:

• Pricing.

• Ticket inventory.

• Status transitions.

• Registration rules.

• Refund calculations.

• Validation.

• Idempotency.

Integration tests

Test:

• Checkout creation.

• Payment webhook.

• Registration creation.

• Refund flow.

• Admin authorization.

• Inventory reservation.

End-to-end tests

Critical journey:

```
Browse event
→ Open event
→ Select ticket
→ Checkout
→ Payment test
→ Confirmation
→ Registration lookup
```

Admin journey:

```
Login
→ Create event
→ Add ticket
→ Publish
→ Verify public event
→ View registration
```

---

52. Acceptance Criteria

Event publishing

Given a complete event: - Admin can publish it. - Event appears publicly. - Event appears in sitemap. - Event can be purchased if on sale.

Ticket purchase

Given available inventory: - User can select quantity. - Total is calculated correctly. - Payment session is created. - Successful webhook marks order paid. - Registration is created once. - Confirmation is sent.

Sold out

Given zero inventory: - Purchase CTA is disabled. - User cannot create a successful checkout. - Event displays Sold Out. - Waitlist can be enabled.

Payment webhook

Given duplicate webhook: - Only one payment is recorded. - Only one registration is created.

Cancellation

Given a cancelled event: - New purchases are blocked. - Public page

shows cancellation state. - Admin can notify attendees. - Refund workflow is available according to policy.

---

53. MVP Scope

Phase 1 — Foundation

• Next.js project.

• TypeScript.

• Prisma.

• PostgreSQL.

• Authentication.

• RBAC.

• Base layout.

• Environment configuration.

• Logging.

Phase 2 — Public Website

• Homepage.

• About.

• Events.

• Event detail.

• FAQ.

• Contact.

• Legal pages.

• SEO.

• Responsive UI.

Phase 3 — Admin

• Dashboard.

• Events.

• Locations.

• Instructors.

• FAQs.

• Content.

• Ticket management.

Phase 4 — Commerce

• Checkout.

• Orders.

• Payment provider.

• Webhooks.

• Registration.

• Confirmation emails.

• Refunds.

Phase 5 — Operations

• Attendee management.

• Reports.

• Export.

• Reminders.

• Audit logs.

• Check-in foundation.

---

54. Future Increments

Increment 1 — Advanced ticketing

• Early-bird pricing.

• VIP tickets.

• Group tickets.

• Coupon codes.

• Ticket transfers.

• Waitlists.

Increment 2 — Attendee portal

• Account creation.

• Registration history.

• Download ticket.

• Update profile.

• Event resources.

Increment 3 — Check-in

• QR code.

• Mobile scanner.

• Real-time attendee validation.

• Attendance reporting.

Increment 4 — Courses

• Online courses.

• Video lessons.

• Quizzes.

• Progress.

• Certificates.

Increment 5 — Membership

• Membership tiers.

• Member pricing.

• Exclusive events.

• Recurring billing.

Increment 6 — Multi-organizer platform

If the business expands:

```
Platform
■■■ Organization A
■■■ Organization B
■■■ Organization C
```

Each organization can have: - Branding. - Events. - Instructors. -

Locations. - Payment configuration. - Admin users.

This should not be implemented in MVP but the domain model should avoid

hard-coded assumptions that make it impossible later.

---

55. Migration from Wix

Migration should happen in controlled stages.

Step 1

Inventory existing content:

• Pages.

• Events.

• Event descriptions.

• Images.

• FAQs.

• Instructor information.

• Legal content.

Step 2

Create database records.

Step 3

Upload media to new storage.

Step 4

Create redirects from old URLs where possible.

Example:

```
/event-details/old-event-slug
→
/events/new-event-slug
```

Step 5

Verify SEO metadata.

Step 6

Run payment test.

Step 7

Run complete registration test.

Step 8

Launch.

---

56. URL Strategy

Prefer clean URLs:

```
/events/tour-1-foundations-everyday-herbal-remedies
/events/vol-2-herbal-healing-everyday-wellness
/events/vol-3-advanced-herbal-formulations
```

Do not expose database IDs in public URLs.

---

57. SEO Migration

Preserve search value by:

• Keeping meaningful event titles.

• Creating canonical URLs.

• Adding redirects.

• Generating structured data.

• Preserving metadata.

• Generating XML sitemap.

• Using descriptive image alt text.

• Avoiding duplicate event URLs.

---

58. Analytics

MVP events:

```
page_view
event_view
ticket_selection
checkout_started
checkout_completed
payment_failed
registration_completed
```

Recommended funnel:

```
Event View
↓
Ticket Selection
↓
Checkout Started
↓
```

```
Payment
↓
Registration Confirmed
```

This allows the business to identify checkout conversion problems.

---

59. Admin Reporting

Minimum reports:

Event report

• Capacity.

• Tickets sold.

• Tickets remaining.

• Revenue.

• Refunds.

• Registrations.

• Attendance.

Revenue report

• Gross sales.

• Discounts.

• Refunds.

• Net collected.

• By event.

• By date.

Attendee report

• Registrations.

• Paid status.

• Attendance.

• No-shows.

Exports should support CSV.

---

60. Operational Rules

1. Never confirm registration from the browser redirect alone.

2. Payment webhook is authoritative.

3. Never allow inventory to become negative.

4. Never store card numbers.

5. Never expose internal database IDs publicly where avoidable.

6. Every admin mutation must be authorized.

7. Important mutations must be auditable.

8. Public event content must be publishable without developer intervention.

9. Legal acknowledgement must be versioned.

10. Email failure must not reverse a successful payment.

11. Duplicate webhooks must be safe.

12. Refund state must remain consistent with payment-provider state.

---

61. Recommended Project Structure

```
herbal-health/
■■■ prisma/
■ ■■■ schema.prisma
■ ■■■ migrations/
■ ■■■ seed.ts
■■■ public/
■ ■■■ ...
■■■ src/
■ ■■■ app/
■ ■ ■■■ (public)/
■ ■ ■ ■■■ page.tsx
■ ■ ■ ■■■ events/
■ ■ ■ ■■■ about/
■ ■ ■ ■■■ faq/
■ ■ ■ ■■■ contact/
■ ■ ■■■ admin/
■ ■ ■■■ api/
■ ■ ■ ■■■ webhooks/
■ ■ ■■■ checkout/
■ ■■■ components/
■ ■■■ features/
■ ■ ■■■ events/
■ ■ ■■■ checkout/
■ ■ ■■■ registrations/
■ ■ ■■■ attendees/
■ ■ ■■■ admin/
■ ■■■ lib/
■ ■ ■■■ auth/
■ ■ ■■■ db/
■ ■ ■■■ email/
■ ■ ■■■ payments/
■ ■ ■■■ storage/
■ ■ ■■■ validation/
■ ■■■ server/
■ ■■■ services/
■ ■■■ repositories/
■ ■■■ jobs/
■■■ tests/
■■■ .env.example
■■■ package.json
■■■ README.md
```

---

62. Development Sequence

Recommended implementation order:

Sprint 1

Architecture, database, authentication, admin shell.

Sprint 2

Public website and CMS foundations.

Sprint 3

Events, locations, instructors, FAQs.

Sprint 4

Tickets, inventory and checkout.

Sprint 5

Payment integration and webhooks.

Sprint 6

Registration, email, attendee management.

Sprint 7

Reports, audit logs, SEO, analytics.

Sprint 8

QA, security review, migration and launch.

---

63. Definition of Done

The MVP is ready for production when:

• Public website is responsive.

• All required pages work.

• Admin can create and publish events.

• Ticket inventory is accurate.

• Checkout works.

• Payment provider is verified in production.

• Webhooks are signature-verified.

• Duplicate webhooks are idempotent.

• Successful payments create registrations.

• Confirmation email works.

• Refund workflow works.

• Legal acknowledgement is recorded.

• Admin authorization is tested.

• No critical security issues remain.

• SEO metadata and sitemap work.

• Production backups are configured.

• Error monitoring is configured.

• Wix content has been migrated.

• Old URLs have redirects where required.

• End-to-end registration test passes.

---

64. Key Product Decision

The rebuild should **not** be implemented as a visual clone of the

existing Wix site.

The existing website is the reference for:

• Content.

• Branding.

• User journey.

• Event model.

• Ticketing behavior.

The new application should become the system of record for:

• Events.

• Tickets.

• Inventory.

• Orders.

• Payments.

• Registrations.

• Attendees.

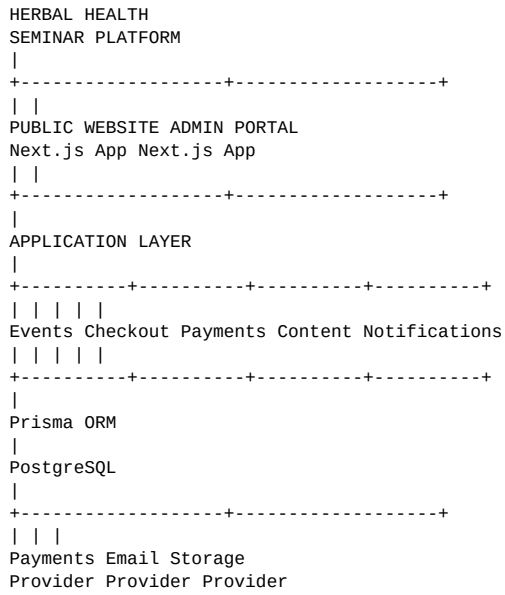
• Content.

• Notifications.

This makes the product easier to operate today and gives the organization a foundation for courses, memberships, certificates, online education, and potentially a multi-organizer event platform later.

---

65. Final Architecture Summary



Recommended MVP principle: keep the architecture clean and extensible, but do not overbuild features that the current business does not need. The first release should be exceptionally strong at one core workflow: **publish seminar → sell tickets → collect payment → confirm attendee → manage event**.